MoMo API Technical Report

1. Introduction to API Security

In this project, we implemented a REST API for accessing mobile money transaction data. Security is a critical consideration for any API that exposes financial information. Our API uses **Basic Authentication** to protect endpoints:

- **How it works:** Each request must include a username and password encoded in Base64. The server checks credentials before returning data.
- **Limitations of Basic Auth:**
    - Credentials are sent in **every request** and only Base64-encoded, not encrypted.
    - Vulnerable to **eavesdropping** if HTTPS is not used.
    - No session management or token expiration.
- **Stronger alternatives**:
    - **JWT (JSON Web Tokens):** Stateless, can include expiry and user roles.
    - **OAuth2:** Standard for authorization; allows limited access to resources without sharing credentials.

> Our implementation is sufficient for demonstration but would require HTTPS and more robust auth in production.

2. Documentation of Endpoints

Base URL: `http://localhost:8500`

Authentication: Basic Auth required with valid username/password (e.g., `admin:password123`).

2.1 GET /transactions

Description: Retrieve all SMS transaction records.

Request Example:

GET http://localhost:8500/transactions
Authorization: Basic {base64-encoded credentials}

Response Example:


```
{
 "success": true,
 "count": 10,
 "transactions": [
   {
     "id": "1f1e88",
     "transactionType": 1,
     "amount": 5500.0,
     "sender": "M-Money",
     "receiver": "Gasasira",
     "createdAt": "2026-01-28T15:24:01.916912",
     "body": "TxId: custom12345. Your payment of 5500 RWF to Gasasira..."
   }
 ]
}
```


Error Codes:

- `401 Unauthorized` – Missing or invalid credentials
- `404 Not Found` – No transactions available

2.2 GET /transactions/{id}

Description: Retrieve a single transaction by short ID.

Request Example:

GET http://localhost:8500/transactions/1f1e88
Authorization: Basic {base64-encoded credentials}

Response Example (200):


```
{
 "id": "1f1e88",
 "_full_id": "1f1e886d-2352-4a6c-8586-40ffdee85d0f",
 "transactionType": 1,
 "amount": 5500.0,
 "sender": "M-Money",
 "receiver": "Gasasira",
 "createdAt": "2026-01-28T15:24:01.916912",
 "body": "TxId: custom12345. Your payment of 5500 RWF to Gasasira..."
}
```

Response Example (404):


```
{
 "error": true,
 "message": "Transaction 1f1e88 not found"
}
```


2.3 POST /transactions

Description: Add a new transaction (ID auto-generated by the system).

Request Example:


```
{
 "transactionType": 1,
 "amount": 5000,
 "sender": "M-Money",
 "receiver": "Alice Smith",
 "body": "TxId: custom12345. Your payment of 5000 RWF to Alice Smith..."
}
```


Response Example (201 Created):


```
{
 "id": "2f26bd",
 "_full_id": "2f26bd12-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
 "transactionType": 1,
 "amount": 5000,
 "sender": "M-Money",
 "receiver": "Alice Smith",
 "createdAt": "2026-01-28T15:07:58.122817",
 "body": "TxId: custom12345. Your payment of 5000 RWF to Alice Smith..."
}
```


2.4 PUT /transactions/{id}

Description: Update any field of an existing transaction.

Request Example:


```
{
 "amount": 5500,
 "receiver": "Gasasira"
}
```


Response Example (200):


```
{
 "id": "1f1e88",
```

"_full_id": "1f1e886d-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
"transactionType": 1,
"amount": 5500,
"sender": "M-Money",
"receiver": "Gasasira",
"updatedAt": "2026-01-28T15:30:42.441708",
"body": "TxId: custom12345. Your payment of 5500 RWF to Gasasira..."
}

2.5 DELETE /transactions/{id}

Description: Delete a transaction by short ID.

Request Example:

DELETE http://localhost:8500/transactions/1f1e88
Authorization: Basic {base64-encoded credentials}

Response Example (200):

{
"success": true,
"message": "Deleted",
"transaction": {
"id": "1f1e88",
"amount": 5500,
"receiver": "Gasasira"
}
}

Error Responses

- `401 Unauthorized` – Authentication required
- `404 Not Found` – Transaction not found
- `400 Bad Request` – Missing required fields

3. Results of DSA Comparison

We tested **two methods** for retrieving transactions:

1. **Linear Search** – Scans the transaction list sequentially

```
def linear_search(transactions, search_id):
  for t in transactions:
```

```
        if t["id"] == search_id:
            return t
    return None
```

2. **Dictionary Lookup** – Stores transactions in a dictionary for O(1) average lookup

```
tx_dict = {t["id"]: t for t in transactions}
res = tx_dict.get(search_id)
```

Performance Results (sample of 20 transactions):

Method	Time (s)	Notes
Linear Search	0.000123	Increases linearly with list size
Dict Lookup	0.000001	Constant time lookup, much faster

Reflection:

- Dictionary lookup is faster because it uses **hashing**.
- Linear search requires checking each element sequentially.
- For very large datasets, a **database index** or a **binary search tree** could further improve efficiency.

4. Reflection on Basic Auth Limitations

- Basic Auth is simple but **not secure without HTTPS**.
- Exposes credentials in every request.
- Lacks session management and token expiration.
- In production, **JWT or OAuth2** would be recommended.

Using a short UUID for transactions improves usability for testing but does **not change the security**. Always combine with HTTPS.